

```

diff --git a/.github/workflows/deploy-testnet.yml b/.github/workflows/deploy-
testnet.yml
new file mode 100644
index 00000000..5684747
--- /dev/null
+++ b/.github/workflows/deploy-testnet.yml
@@ -0,0 +1,131 @@
+name: Deploy TRON Contracts (Testnet)
+
+# Manually-triggered deployment of the full D3R-AC TRON contract suite
+# to Shasta or Nile via TronBox, run entirely inside a GitHub Actions
+# runner rather than a developer's own machine or an AI assistant's
+# sandbox. Two reasons this exists as a workflow rather than a local
+# `tronbox migrate`:
+#
+# 1. Network access: TronGrid (api.shasta.trongrid.io /
+#    nile.trongrid.io) needs to be reachable from wherever this runs.
+#    GitHub-hosted runners have normal outbound internet access.
+# 2. Key custody: the deploy private key lives ONLY in this repo's
+#    encrypted GitHub Actions secrets (Settings -> Secrets and
+#    variables -> Actions). It is injected as an environment variable
+#    at run time, never printed, never committed, and never visible
+#    to anyone reading workflow logs -- GitHub automatically redacts
+#    registered secret values that appear in log output. This is the
+#    standard way to keep a signing key out of chat transcripts, out
+#    of a shared sandbox, and out of shell history, while still
+#    letting deployment be triggered on demand and reviewed after the
+#    fact via the run's log.
+#
+# Required repo secrets (add under Settings -> Secrets and variables ->
+# Actions -> "New repository secret") before running this:
+#   TRON_PRIVATE_KEY_SHASTA    deploy wallet's private key, Shasta only
+#   TRON_PRIVATE_KEY_NILE      deploy wallet's private key, Nile only
+#                               (only the one matching the network you
+#                               pick below needs to actually be set)
+#   MULTISIG_OWNERS             comma-separated TRON addresses, e.g.
+#                               "TAbc...,TDef...,TGhi..."
+#   MULTISIG_THRESHOLD          confirmations required, e.g. "2"
+#
+# Before running:
+# 1. Generate a fresh deploy wallet you don't use for anything else
+#    (see docs/deployment-guide.md). Never reuse a real-funds wallet.
+# 2. Fund ONLY that wallet with testnet TRX:
+#    Shasta faucet: https://www.trongrid.io/shasta

```

```

+# Nile faucet: https://nileex.io/join/getJoinPage
+# 3. Add the four secrets above in the GitHub UI. GitHub encrypts
+# these at rest; they are not readable again through the UI or
+# API once saved, only overwritten.
+# 4. Run this workflow from the Actions tab ("Run workflow"), pick
+# the network, and watch the log -- deployed addresses print at
+# the end (that output is NOT secret and is fine to share/record
+# per docs/deployment-guide.md's post-deployment step).
+#
+# What this does NOT do: it does not fund the wallet, generate a key
+# for you, or click "confirm" on behalf of other MultiSigAdmin owners
+# if MULTISIG_THRESHOLD > 1 -- the migration script
+# (tronbox/migrations/2_deploy_d3rac.js) prints exactly what's still
+# needed from the other owners in that case, in this run's log.
+
+on:
+  workflow_dispatch:
+    inputs:
+      network:
+        description: "TRON testnet to deploy to"
+        required: true
+        type: choice
+        options:
+          - shasta
+          - nile
+        default: shasta
+      confirm_intent:
+        description: 'Type "deploy" to confirm — this sends real (testnet)
transactions'
+        required: true
+        type: string
+
+jobs:
+  deploy:
+    name: Deploy to ${ github.event.inputs.network }
+    runs-on: ubuntu-latest
+    # Manual approval gate: requires a GitHub Environment named
+    # "tron-testnet-deploy" to exist with at least one required
+    # reviewer configured (Settings -> Environments). If that
+    # environment doesn't exist yet, GitHub runs the job without a
+    # gate -- create the environment first if you want a second human
+    # to have to click "Approve" before any transaction is sent, on
+    # top of the confirm_intent check below.
+    environment: tron-testnet-deploy
+    defaults:
+      run:
+        working-directory: contracts/tron/tronbox

```

```

+
+   steps:
+     - name: Require explicit confirmation
+       run: |
+         if [ "${{ github.event.inputs.confirm_intent }}" != "deploy" ]; then
+           echo "::error::confirm_intent must be exactly 'deploy' to proceed.
Got: '${{ github.event.inputs.confirm_intent }}'"
+           exit 1
+         fi
+
+     - name: Checkout repository
+       uses: actions/checkout@v4
+
+     - name: Set up Node.js
+       uses: actions/setup-node@v4
+       with:
+         node-version: "22"
+
+     - name: Install TronBox project dependencies
+       run: npm install
+
+     - name: Compile contracts
+       run: npx tronbox compile
+
+     - name: Confirm deploy wallet has a balance before spending gas
+       env:
+         TRON_PRIVATE_KEY_SHASTA: ${ secrets.TRON_PRIVATE_KEY_SHASTA }
+         TRON_PRIVATE_KEY_NILE: ${ secrets.TRON_PRIVATE_KEY_NILE }
+       run: |
+         node ./scripts/check-deploy-balance.js "${{ github.event.inputs.network
+ }}"
+
+     - name: Run migration (deploy + wire the Hub + propose MultiSigAdmin as its
admin)
+       env:
+         TRON_PRIVATE_KEY_SHASTA: ${ secrets.TRON_PRIVATE_KEY_SHASTA }
+         TRON_PRIVATE_KEY_NILE: ${ secrets.TRON_PRIVATE_KEY_NILE }
+         MULTISIG_OWNERS: ${ secrets.MULTISIG_OWNERS }
+         MULTISIG_THRESHOLD: ${ secrets.MULTISIG_THRESHOLD }
+       run: |
+         npx tronbox migrate --network "${{ github.event.inputs.network }}" --
reset
+
+     - name: Summarize deployed addresses as a job artifact
+       if: always()
+       run: |
+         echo "Deployment log above is the source of truth for addresses."

```

```

+         echo "Copy the '=== D3R-AC deployment complete ===' block into"
+         echo "docs/deployment-guide.md or a release note, per that doc's"
+         echo "post-deployment step -- this workflow does not do that commit"
+         echo "automatically, since it's a documentation decision, not a"
+         echo "deployment one."
diff --git a/contracts/tron/test/D3RACTokenFactory.test.mjs
b/contracts/tron/test/D3RACTokenFactory.test.mjs
new file mode 100644
index 0000000..0198034
--- /dev/null
+++ b/contracts/tron/test/D3RACTokenFactory.test.mjs
@@ -0,0 +1,71 @@
+import { expect } from "chai";
+import { ethers, deploy } from "../helpers.mjs";
+import { ethers as ethersLib } from "ethers";
+
+
+describe("D3RACTokenFactory (CREATE2 vanity-address deployment)", function () {
+  it("on-chain prediction, off-chain prediction, and the real deployed address
all agree", async function () {
+    const [deployer, owner] = await ethers.getSigners();
+
+
+    const factory = await deploy("D3RACTokenFactory", deployer);
+    const factoryAddress = await factory.getAddress();
+
+
+    const salt = ethersLib.id("d3rac-test-salt-1"); // any bytes32; a real grind
picks this for its address, not its meaning
+    const initialSupply = 1_000_000n; // pre-scaling; D3RACToken multiplies by
10**decimals internally
+    const ownerAddress = owner.address;
+
+
+    // 1) On-chain prediction, from the factory contract itself.
+    const onChainPrediction = await factory.predictTokenAddress(salt,
initialSupply, ownerAddress);
+
+
+    // 2) Off-chain prediction, using the exact same formula
+    //   scripts/find-vanity-create2-salt.js grinds against - no
+    //   network call, no private key, just public bytecode + math.
+    const D3RACTokenFactory_ = await ethers.getContractFactory("D3RACToken");
+    const initCode = ethersLib.concat([
+      D3RACTokenFactory_.bytecode,
+      D3RACTokenFactory_.interface.encodeDeploy([initialSupply, ownerAddress]),
+    ]);
+    const initCodeHash = ethersLib.keccak256(initCode);
+    const offChainPrediction = ethersLib.getCreate2Address(factoryAddress, salt,
initCodeHash);
+
+
+

```

```

expect(onChainPrediction.toLowerCase()).to.equal(offChainPrediction.toLowerCase());
+
+   // 3) The real deployment. If prediction #1 and #2 are correct, this
+   //   must land exactly on that same address.
+   const tx = await factory.connect(deployer).deployToken(salt, initialSupply,
ownerAddress);
+   const receipt = await tx.wait();
+   const parsed = receipt.logs
+     .map((l) => {
+     try {
+       return factory.interface.parseLog(l);
+     } catch {
+       return null;
+     }
+   })
+     .find((l) => l?.name === "D3RACTokenDeployed");
+
+   expect(parsed).to.not.equal(undefined);
+   const deployedAddress = parsed.args.tokenAddress;
+
+
expect(deployedAddress.toLowerCase()).to.equal(onChainPrediction.toLowerCase());
+
expect(deployedAddress.toLowerCase()).to.equal(offChainPrediction.toLowerCase());
+
+   // Sanity check the deployed contract is a real, working D3RACToken
+   // at that exact predicted address – not just a matching address
+   // with nothing meaningful behind it.
+   const token = D3RACTokenFactory_.attach(deployedAddress);
+   const decimals = await token.decimals();
+   expect(await token.owner()).to.equal(ownerAddress);
+   expect(await token.totalSupply()).to.equal(initialSupply * 10n ** decimals);
+   expect(await token.balanceOf(ownerAddress)).to.equal(initialSupply * 10n **
decimals);
+ });
+
+ it("the same salt can't be used twice from the same factory (CREATE2
semantics)", async function () {
+   const [, owner] = await ethers.getSigners();
+   const factory = await deploy("D3RACTokenFactory", owner);
+
+   const salt = ethersLib.id("d3rac-reuse-test");
+
+   await factory.deployToken(salt, 1000n, owner.address);
+   await expect(factory.deployToken(salt, 1000n,
owner.address)).to.be.revert(ethers);
+ });

```

```

+});
diff --git a/contracts/tron/tronbox/contracts/D3RACTokenFactory.sol
b/contracts/tron/tronbox/contracts/D3RACTokenFactory.sol
new file mode 100644
index 0000000..d96ab61
--- /dev/null
+++ b/contracts/tron/tronbox/contracts/D3RACTokenFactory.sol
@@ -0,0 +1,75 @@
+// SPDX-License-Identifier: MIT
+pragma solidity ^0.8.20;
+
+import "../D3RACToken.sol";
+
+/// @title D3RACTokenFactory
+/// @notice Deploys D3RACToken via CREATE2 (Solidity's `new{salt: ...}`)
+///         instead of a plain `new D3RACToken(...)`, so the token's own
+///         contract address – its permanent, public, on-chain identity –
+///         can be chosen in advance to visibly contain "D3RAC", rather
+///         than landing wherever a plain sequential deployment happens
+///         to put it.
+///
+/// Why this is safe to grind for, with zero private-key involvement:
+/// a CREATE2 address is a pure function of three PUBLIC values –
+/// `address(this)` (this factory's own address, fixed once the factory
+/// itself is deployed), `salt` (any bytes32 the caller picks), and
+/// `keccak256(init_code)` (deterministic from D3RACToken's bytecode +
+/// constructor args). None of that requires a private key to compute
+/// or search over – see scripts/find-vanity-create2-salt.js, which
+/// grinds candidate salts entirely off-chain against this exact
+/// formula. Only the *deploy* step at the end needs a real transaction
+/// (and therefore a signer) – the address is already known before
+/// that transaction is sent.
+///
+/// TRON-specific note (see contracts/tron/README.md's "Deploying a
+/// vanity D3RAC address" section for the full writeup, and
+/// https://developers.tron.network/docs/migrating-eth-contracts-to-tron):
+/// TRON's own raw CREATE2 opcode uses a TRON-specific 0x41 address
+/// prefix internally, but the TRON-Solidity compiler emulates
+/// standard Ethereum behavior for the high-level `new{salt: ...}`
+/// syntax used below – it computes the address with the standard
+/// 0xff-prefixed EVM formula, then that 20-byte EVM-style address
+/// becomes a real TRON address once prefixed with 0x41 and
+/// base58check-encoded (exactly what TronWeb's `address.fromHex` does,
+/// and what the salt-grinding script accounts for). This contract
+/// does not touch the raw `create2` opcode directly, so this is the
+/// path that applies.
+contract D3RACTokenFactory {

```

```

+     event D3RACTokenDeployed(address indexed tokenAddress, bytes32 indexed salt,
address indexed owner);
+
+     /// @notice Deploys a D3RACToken at the CREATE2 address determined by
+     ///         `salt` + this factory's own address + D3RACToken's
+     ///         bytecode and constructor args. Reverts if a contract is
+     ///         already deployed at that address (CREATE2 semantics -
+     ///         one salt can only ever be used once from this factory).
+     /// @param salt Chosen off-chain by scripts/find-vanity-create2-salt.js
+     ///         to land on an address containing the desired substring.
+     /// @param initialSupply Passed straight through to D3RACToken's constructor.
+     /// @param owner_ Passed straight through to D3RACToken's constructor -
+     ///         this becomes the new token's `owner` immediately (not the
+     ///         factory), so the factory holds no ongoing power over it.
+     function deployToken(bytes32 salt, uint256 initialSupply, address owner_)
external returns (address tokenAddress) {
+         D3RACToken token = new D3RACToken{salt: salt}(initialSupply, owner_);
+         tokenAddress = address(token);
+         emit D3RACTokenDeployed(tokenAddress, salt, owner_);
+     }
+
+     /// @notice Predicts the address `deployToken` will produce for a given
+     ///         `salt`, without deploying anything - an on-chain
+     ///         cross-check for the off-chain grinding script's own
+     ///         prediction (see that script's own EVM CREATE2 formula).
+     ///         Correct once this factory's own address is final (i.e.
+     ///         after this contract itself has been deployed for real).
+     function predictTokenAddress(bytes32 salt, uint256 initialSupply, address
owner_) external view returns (address predicted) {
+         bytes memory initCode = abi.encodePacked(
+             type(D3RACToken).creationCode,
+             abi.encode(initialSupply, owner_)
+         );
+         bytes32 initCodeHash = keccak256(initCode);
+         predicted = address(
+             uint160(uint256(keccak256(abi.encodePacked(bytes1(0xff),
address(this), salt, initCodeHash))))
+         );
+     }
+}

diff --git a/contracts/tron/tronbox/package-lock.json
b/contracts/tron/tronbox/package-lock.json
index 7e48d5b..9d270a0 100644
--- a/contracts/tron/tronbox/package-lock.json
+++ b/contracts/tron/tronbox/package-lock.json
@@ -8,7 +8,216 @@
     "name": "d3rac-tron-contracts-tronbox",

```

```

    "version": "0.2.0",
    "dependencies": {
-       "dotenv": "^16.4.5"
+       "dotenv": "^16.4.5",
+       "ethers": "^6.17.0",
+       "tronweb": "^6.5.0"
+     }
+   },
+   "node_modules/@adraffy/ens-normalize": {
+     "version": "1.11.1",
+     "resolved": "https://registry.npmjs.org/@adraffy/ens-normalize/-/ens-normalize-1.11.1.tgz",
+     "integrity": "sha512-nhCBV3quEgesuf7c7KYfperqSS14T8bYuvJ8PcLJp6znkZpFc0AuW4qBtr8eKVyPpe/8RSr7sglCWPU5eax
+     "license": "MIT"
+   },
+   "node_modules/@noble/curves": {
+     "version": "1.2.0",
+     "resolved": "https://registry.npmjs.org/@noble/curves/-/curves-1.2.0.tgz",
+     "integrity": "sha512-oYclrNgRaM9SsBUBVbb8M6DTV7ZHRTKugureoYEncY5c65HOMRzvSiTE3y5CYaPYJA/GVkrhXEoF0M3Ya9F
+     "license": "MIT",
+     "dependencies": {
+       "@noble/hashes": "1.3.2"
+     },
+     "funding": {
+       "url": "https://paulmillr.com/funding/"
+     }
+   },
+   "node_modules/@noble/hashes": {
+     "version": "1.3.2",
+     "resolved": "https://registry.npmjs.org/@noble/hashes/-/hashes-1.3.2.tgz",
+     "integrity": "sha512-MVC8EAQp7MvEcm30KWENFjgR+Mkmf+D189XJTkFilwohU5hcBbn1ZkKq7KVTi2Hme3PMGF390DaL52beVrI
+     "license": "MIT",
+     "engines": {
+       "node": ">= 16"
+     },
+     "funding": {
+       "url": "https://paulmillr.com/funding/"
+     }
+   },
+   "node_modules/@scure/base": {
+     "version": "1.1.9",
+     "resolved": "https://registry.npmjs.org/@scure/base/-/base-1.1.9.tgz",
+     "integrity": "sha512-8YKh18GHiNI/pU2VMAofa2Tor7PJRAjwQLBBuilkJ9L5+13yVbC7JO/wS7piioAvPSwR3JKM1IJ/u4xQzbc

```



```

+     "license": "MIT",
+     "funding": {
+       "url": "https://paulmillr.com/funding/"
+     }
+   },
+   "node_modules/@scure/bip32": {
+     "version": "1.4.0",
+     "resolved": "https://registry.npmjs.org/@scure/bip32/-/bip32-1.4.0.tgz",
+     "integrity": "sha512-sVUpc0Vq3tXCkDGYVWGIZTRfnvu8LoTdaev7vbwh0omSvVORONr960MQWdKqJDCReIEmTj3PAr7303aoxz7
+     "license": "MIT",
+     "dependencies": {
+       "@noble/curves": "~1.4.0",
+       "@noble/hashes": "~1.4.0",
+       "@scure/base": "~1.1.6"
+     },
+     "funding": {
+       "url": "https://paulmillr.com/funding/"
+     }
+   },
+   "node_modules/@scure/bip32/node_modules/@noble/curves": {
+     "version": "1.4.2",
+     "resolved": "https://registry.npmjs.org/@noble/curves/-/curves-1.4.2.tgz",
+     "integrity": "sha512-TavHr8qycMChk8UwMld0ZDRvatedkzWfH8IiaeGCfymOP5i0hSCozz9vHOL0nkwk7HRMlFnAiKpS2jrUmSy
+     "license": "MIT",
+     "dependencies": {
+       "@noble/hashes": "1.4.0"
+     },
+     "funding": {
+       "url": "https://paulmillr.com/funding/"
+     }
+   },
+   "node_modules/@scure/bip32/node_modules/@noble/hashes": {
+     "version": "1.4.0",
+     "resolved": "https://registry.npmjs.org/@noble/hashes/-/hashes-1.4.0.tgz",
+     "integrity": "sha512-V1JJlWTRUqHHrOSh597hURcMqVKVGL/ea3kv0gSnEdsEZ0/+VyPghM1lMNGc00z7CIQorSvbKpuJkxvuHbv
+     "license": "MIT",
+     "engines": {
+       "node": ">= 16"
+     },
+     "funding": {
+       "url": "https://paulmillr.com/funding/"
+     }
+   },
+   "node_modules/@scure/bip39": {

```

```

+     "version": "1.3.0",
+     "resolved": "https://registry.npmjs.org/@scure/bip39/-/bip39-1.3.0.tgz",
+     "integrity": "sha512-
disdg7gHuTDZtY+ZdkmLpPCk7fxZSu3gBiEGuoC1XYxv9cGx3Z6cpTggCgW6odSOOIXCiDjuGejW+aJKCY/
+     "license": "MIT",
+     "dependencies": {
+       "@noble/hashes": "~1.4.0",
+       "@scure/base": "~1.1.6"
+     },
+     "funding": {
+       "url": "https://paulmillr.com/funding/"
+     }
+   },
+   "node_modules/@scure/bip39/node_modules/@noble/hashes": {
+     "version": "1.4.0",
+     "resolved": "https://registry.npmjs.org/@noble/hashes/-/hashes-1.4.0.tgz",
+     "integrity": "sha512-
V1JJ1WTRUqHHrOSH597hURcMqVKVGL/ea3kv0gSnEdsEZ0/+VyPghM1lMNGc00z7CIQorSvbKpuJkxvuHbv
+     "license": "MIT",
+     "engines": {
+       "node": ">= 16"
+     },
+     "funding": {
+       "url": "https://paulmillr.com/funding/"
+     }
+   },
+   "node_modules/@types/node": {
+     "version": "22.7.5",
+     "resolved": "https://registry.npmjs.org/@types/node/-/node-22.7.5.tgz",
+     "integrity": "sha512-
jML7s2NAzMWc//QSJ1a3prpk78cOPchGvXJsC3C6R6PSMoooztvRVQEz89gmBTBY1SPMaqo5teB4uNHPdet
+     "license": "MIT",
+     "dependencies": {
+       "undici-types": "~6.19.2"
+     }
+   },
+   "node_modules/aes-js": {
+     "version": "4.0.0-beta.5",
+     "resolved": "https://registry.npmjs.org/aes-js/-/aes-js-4.0.0-beta.5.tgz",
+     "integrity": "sha512-
G965FqalsNyrPqgEGON7nIx1e/OVENSgiEIzyC63haUMuvNnwIgIjMs52h1TCKhkBny7A2ORNlfY9Zu+jmC
+     "license": "MIT"
+   },
+   "node_modules/agent-base": {
+     "version": "6.0.2",
+     "resolved": "https://registry.npmjs.org/agent-base/-/agent-base-6.0.2.tgz",
+     "integrity": "sha512-

```

```

RZNwNclF7+MS/8bDg70amg32dyeZGZxiDuQmZxKLAlQjr3jGyLx+4Kkk58UO7D2QdgFIQCovuSuZESne6RG
+   "license": "MIT",
+   "dependencies": {
+     "debug": "4"
+   },
+   "engines": {
+     "node": ">= 6.0.0"
+   }
+ },
+ "node_modules/asynckit": {
+   "version": "0.4.0",
+   "resolved": "https://registry.npmjs.org/asynckit/-/asynckit-0.4.0.tgz",
+   "integrity": "sha512-
Oei9OH4tRh0YqU3GxhX79dM/mwVgvbZJaSNaRk+bshkj0S5cfHcgYakreBjrHwatXKbz+IoIdYLxrKim2Mj
+   "license": "MIT"
+ },
+ "node_modules/axios": {
+   "version": "1.18.0",
+   "resolved": "https://registry.npmjs.org/axios/-/axios-1.18.0.tgz",
+   "integrity": "sha512-
E32NzpYKp++W7XRe52rHiXV2ehxmh3wbdg07MHeFM+vqxLBYHzt0ElkiImtOBxtOmy0yoC8C6uESVV84Y2
+   "license": "MIT",
+   "dependencies": {
+     "follow-redirects": "^1.16.0",
+     "form-data": "^4.0.5",
+     "https-proxy-agent": "^5.0.1",
+     "proxy-from-env": "^2.1.0"
+   }
+ },
+ "node_modules/bignumber.js": {
+   "version": "9.1.2",
+   "resolved": "https://registry.npmjs.org/bignumber.js/-/bignumber.js-
9.1.2.tgz",
+   "integrity": "sha512-
2/mKyZH9K85bzOEfhXDBFZTGd1CTs+5IHpeFQo9luiBG7hghdC851Pj2WAhb6E3R6b9tZj/XKhbg4fum+Ke
+   "license": "MIT",
+   "engines": {
+     "node": "*"
+   }
+ },
+ "node_modules/call-bind-apply-helpers": {
+   "version": "1.0.2",
+   "resolved": "https://registry.npmjs.org/call-bind-apply-helpers/-/call-
bind-apply-helpers-1.0.2.tgz",
+   "integrity": "sha512-
SplablJ0ivDkSzjcaJdxEunN5/XvksFJ2sMBFfq6x0ryhQV/2b/KwFe21cMpmHtPOSij8K99/wSfoEuTObrn
+   "license": "MIT",

```

```
+      "dependencies": {
+        "es-errors": "^1.3.0",
+        "function-bind": "^1.1.2"
+      },
+      "engines": {
+        "node": ">= 0.4"
+      }
+    },
+    "node_modules/combined-stream": {
+      "version": "1.0.8",
+      "resolved": "https://registry.npmjs.org/combined-stream/-/combined-stream-1.0.8.tgz",
+      "integrity": "sha512-FQ4MRfuJeHf7cBbBMJFXhKSDq+2kAArBlmRBvcvFE5BB1HZKXtSFASDhdlz9zOYwxh8lDdnvmMOe/+5cdc
+      "license": "MIT",
+      "dependencies": {
+        "delayed-stream": "~1.0.0"
+      },
+      "engines": {
+        "node": ">= 0.8"
+      }
+    },
+    "node_modules/debug": {
+      "version": "4.4.3",
+      "resolved": "https://registry.npmjs.org/debug/-/debug-4.4.3.tgz",
+      "integrity": "sha512-
+      "license": "MIT",
+      "dependencies": {
+        "ms": "^2.1.3"
+      },
+      "engines": {
+        "node": ">=6.0"
+      },
+      "peerDependenciesMeta": {
+        "supports-color": {
+          "optional": true
+        }
+      }
+    },
+    "node_modules/delayed-stream": {
+      "version": "1.0.0",
+      "resolved": "https://registry.npmjs.org/delayed-stream/-/delayed-stream-1.0.0.tgz",
+      "integrity": "sha512-
+      "license": "MIT",
+      "dependencies": {
+        "ms": "^2.1.3"
+      },
+      "engines": {
+        "node": ">=0.10"
+      }
+    },
+    "node_modules/diff": {
+      "version": "5.2.0",
+      "resolved": "https://registry.npmjs.org/diff/-/diff-5.2.0.tgz",
+      "integrity": "sha512-uIFDpblp0HTUjliWCY7ZPVRDyrg4iQUg7nBzoU3edq4WpXONgaowMFafrhPhe4uW4Fq6joU/iEgJ73xGQU2Yxg==
+      "license": "MIT",
+      "engines": {
+        "node": ">=0.6.0"
+      }
+    },
+    "node_modules/diff-sequences": {
+      "version": "29.6.0",
+      "resolved": "https://registry.npmjs.org/diff-sequences/-/diff-sequences-29.6.0.tgz",
+      "integrity": "sha512-4F39H29W2G0ppD89yFI5gFsiZm9sTl902AcRy1bkgyZR3FVW3EJ/obQN+jAN/2jqcyk1UaEEAq82dBhJW9XNQ==
+      "license": "MIT",
+      "engines": {
+        "node": ">=18"
+      }
+    },
+    "node_modules/dns-packet": {
+      "version": "5.6.1",
+      "resolved": "https://registry.npmjs.org/dns-packet/-/dns-packet-5.6.1.tgz",
+      "integrity": "sha512-r43g4bKQ1k0506S9LkfDoF6XUqK38QwWJD0nW9T0Rl57hE0YogV8biqZuH1eHhOYjp5UWliz9LWtu9Hsg4uKw==
+      "license": "MIT",
+      "dependencies": {
+        "ip-address": "9.1.0",
+        "is-buffer": "2.0.5"
+      },
+      "engines": {
+        "node": ">=12"
+      }
+    },
+    "node_modules/dns-txt": {
+      "version": "6.0.1",
+      "resolved": "https://registry.npmjs.org/dns-txt/-/dns-txt-6.0.1.tgz",
+      "integrity": "sha512-4nWUg6OxOmb8B12tZZYl6YyX39sDoq0Qq8a47XV2FYh6HwT519VNrBIW6ZUjIj14T59PDzFwRZA1PpCZ1jg==
+      "license": "MIT",
+      "dependencies": {
+        "buffer-indexof": "1.1.2"
+      },
+      "engines": {
+        "node": ">=12"
+      }
+    },
+    "node_modules/dotenv": {
+      "version": "16.5.0",
+      "resolved": "https://registry.npmjs.org/dotenv/-/dotenv-16.5.0.tgz",
+      "integrity": "sha512-m/d+0yT8gUz6SeST0ATjNk9z5lWnO69WAIQ5P8Kz07AN3vENqjMtY0F1Ml8vwlG9WZc89n9iwHfQTl7v50xvDQ==
+      "license": "BSD-2-Clause",
+      "engines": {
+        "node": ">=12"
+      }
+    },
+    "node_modules/duplexify": {
+      "version": "4.1.3",
+      "resolved": "https://registry.npmjs.org/duplexify/-/duplexify-4.1.3.tgz",
+      "integrity": "sha512-3Lwz2V7QbSvJtO1uiZK5R7IZgR3n/4p3h4Zrclr/G62t9BpJ41L73I8p3x2rX3dY12LIhRcKX58wX09i66Ag==
+      "license": "MIT",
+      "dependencies": {
+        "duplexify": "4.1.3",
+        "inherits": "2.0.4",
+        "readable-stream": "3.6.0",
+        "stream-equal": "1.1.0",
+        "stream-shift": "1.0.1"
+      }
+    },
+    "node_modules/eastasianwidth": {
+      "version": "0.5.1",
+      "resolved": "https://registry.npmjs.org/eastasianwidth/-/eastasianwidth-0.5.1.tgz",
+      "integrity": "sha512-ckrtO/oKlNv71ln832xYCuYD8iL1BYY1kIlGxS51h/qOZag4l2y6X41UaQig7dyl0uWag60dtdELkQd611l6dA==
+      "license": "MIT"
+    },
+    "node_modules/emoji-regex": {
+      "version": "9.2.2",
+      "resolved": "https://registry.npmjs.org/emoji-regex/-/emoji-regex-9.2.2.tgz",
+      "integrity": "sha512-l53q0h4jkAhf0C/Ah2ddQo6vO/oClvH06/s88KfK271btdoJHAaF7Y1D/C33sPQ51F9EcEiR2gY0U2hY0M0H1A==
+      "license": "MIT"
+    },
+    "node_modules/encoding": {
+      "version": "0.1.13",
+      "resolved": "https://registry.npmjs.org/encoding/-/encoding-0.1.13.tgz",
+      "integrity": "sha512-+Bk9zuwbWVJfG96V8LdI+XlXqnQFp1Zj4tItvK30l1IkQcWwptHw4L445h6/rwTrgNO0i0rAZkeqM6N1pjcLA==
+      "license": "MIT",
+      "dependencies": {
+        "iconv-lite": "0.6.3"
+      }
+    },
+    "node_modules/end-of-stream": {
+      "version": "1.4.4",
+      "resolved": "https://registry.npmjs.org/end-of-stream/-/end-of-stream-1.4.4.tgz",
+      "integrity": "sha512-+p1n2clQ1KpL7R2J8ngvXxoEIOEkCU8hyEo+Q6pbaHn0tStq9DZNpLGqZTQnFzLWReQ8P2h3Fq3tLYdChm07oA==
+      "license": "MIT",
+      "dependencies": {
+        "once": "1.4.0"
+      }
+    },
+    "node_modules/enhanced-resolve": {
+      "version": "5.17.1",
+      "resolved": "https://registry.npmjs.org/enhanced-resolve/-/enhanced-resolve-5.17.1.tgz",
+      "integrity": "sha512-LMH5p2t8k0k+k/HJZu9VWes4+f7YcV3nVtRgoh8XWqBsgJ8ODz8cHvWlXp4t3L3GfV2Lch1rJt9O3o8Y188pDg==
+      "license": "MIT",
+      "dependencies": {
+        "graceful-fs": "4.2.11",
+        "tapable": "2.2.1"
+      },
+      "engines": {
+        "node": ">=10.13.0"
+      }
+    },
+    "node_modules/entities": {
+      "version": "4.5.0",
+      "resolved": "https://registry.npmjs.org/entities/-/entities-4.5.0.tgz",
+      "integrity": "sha512-4Zs38z9Ue9s3jS0rueNkfX8eTsDnFMM9lWEjkCkJbfkzs/qS1K7S0h2TbZPdiY047R/5yxxApL1U9Id7Kz1E8w==
+      "license": "BSD-2-Clause",
+      "engines": {
+        "node": ">=12"
+      }
+    },
+    "node_modules/env-paths": {
+      "version": "3.0.0",
+      "resolved": "https://registry.npmjs.org/env-paths/-/env-paths-3.0.0.tgz",
+      "integrity": "sha512-eyJLWuN8ZTnQmY8In70LjJ9FuTjVnQjW9N1Yd9F8Wb/gWU3UCO7EK+DGNI04Wp0Wo48VTQ1lr5NhXWRW9TKE8g==
+      "license": "MIT",
+      "engines": {
+        "node": ">=10"
+      }
+    },
+    "node_modules/err-code": {
+      "version": "2.0.3",
+      "resolved": "https://registry.npmjs.org/err-code/-/err-code-2.0.3.tgz",
+      "integrity": "sha512-2b2ecDyswfIxLr5Xd507V7BnIcwG1e6yGNSv1/kF7G0eV9H3oTP4X3389r31QF5T3+8o8mlAPm7c306pD60S8g==
+      "license": "MIT"
+    },
+    "node_modules/error-ex": {
+      "version": "1.3.2",
+      "resolved": "https://registry.npmjs.org/error-ex/-/error-ex-1.3.2.tgz",
+      "integrity": "sha512-7dFHNsg1M10Cz81JANjEcTv5Ro2ofYkxoix0ZqLUNwuplECwzouTnZ8R1F0l9O9G4MogXpmv16wQWg2oY1bk8W4=
+      "license": "MIT",
+      "dependencies": {
+        "is-extendable": "1.0.1"
+      }
+    },
+    "node_modules/es-define-property": {
+      "version": "1.0.0",
+      "resolved": "https://registry.npmjs.org/es-define-property/-/es-define-property-1.0.0.tgz",
+      "integrity": "sha512-jx753oUa45999xM0kUk1v9BQ6ZM9YBV4BAgRmdDlKQ2DzWGe3wVN8Qs/67Jt6BvgTKd6VgKR0/9VtKloypI8Q==
+      "license": "MIT",
+      "engines": {
+        "node": ">= 0.4"
+      }
+    },
+    "node_modules/es-errors": {
+      "version": "1.3.0",
+      "resolved": "https://registry.npmjs.org/es-errors/-/es-errors-1.3.0.tgz",
+      "integrity": "sha512-UfV1e+ZHRW8WzcU3e20g16f8pSog/04SEZqy40pfBJr+7yn8eFhSSzv6pAFjEPv0dVP5kUBPh5SclYGa/kz5Uw==
+      "license": "MIT",
+      "engines": {
+        "node": ">=12"
+      }
+    },
+    "node_modules/es-module-lexer": {
+      "version": "1.5.4",
+      "resolved": "https://registry.npmjs.org/es-module-lexer/-/es-module-lexer-1.5.4.tgz",
+      "integrity": "sha512-MVNk2XPlWoYpStQym8I7Gc8PD2Yn8g1dK6822tV1pK1g5OQUW4/bOteFcZs9R0IKST9oFS8JnQa1Gw2WfjUodg==
+      "license": "MIT"
+    },
+    "node_modules/es-object-atoms": {
+      "version": "1.0.0",
+      "resolved": "https://registry.npmjs.org/es-object-atoms/-/es-object-atoms-1.0.0.tgz",
+      "integrity": "sha512-U8WwpoScBV8WqBuSfzmm5Ui8CKYJKz4PHaj1bcu9Z5Dx4QINuzRZRbf55sx7VYkZ9QdG15Ug8Ujg/V+b19cB6w==
+      "license": "MIT",
+      "engines": {
+        "node": ">= 0.4"
+      }
+    },
+    "node_modules/es-set-tostringtag": {
+      "version": "2.0.3",
+      "resolved": "https://registry.npmjs.org/es-set-tostringtag/-/es-set-tostringtag-2.0.3.tgz",
+      "integrity": "sha512-3T
```

```

+     "engines": {
+       "node": ">=0.4.0"
+     },
+     "node_modules/dotenv": {
@@ -22,6 +231,403 @@
+       "funding": {
+         "url": "https://dotenvx.com"
+       }
+     },
+     "node_modules/dunder-proto": {
+       "version": "1.0.1",
+       "resolved": "https://registry.npmjs.org/dunder-proto/-/dunder-proto-
1.0.1.tgz",
+       "integrity": "sha512-
KIN/nDJBQRcXw0MLVhZE9iQHmG68qAVIBg9CqmUYjmQIhgij9U5MFvrqkUL5FbtyyzZuOeOt0zdeRe4UY7c
+       "license": "MIT",
+       "dependencies": {
+         "call-bind-apply-helpers": "^1.0.1",
+         "es-errors": "^1.3.0",
+         "gopd": "^1.2.0"
+       },
+       "engines": {
+         "node": ">= 0.4"
+       }
+     },
+     "node_modules/es-define-property": {
+       "version": "1.0.1",
+       "resolved": "https://registry.npmjs.org/es-define-property/-/es-define-
property-1.0.1.tgz",
+       "integrity": "sha512-
e3nRfgfUZ4rNGL232gUgX06QNYyez04KdjFrF+LTRoOXmrOgFKDg4BCdsjW8EnT69eqdYGmRpJwiPVYNrCa
+       "license": "MIT",
+       "engines": {
+         "node": ">= 0.4"
+       }
+     },
+     "node_modules/es-errors": {
+       "version": "1.3.0",
+       "resolved": "https://registry.npmjs.org/es-errors/-/es-errors-1.3.0.tgz",
+       "integrity": "sha512-
Zf5H2Kxt2xjTvbJvP2ZWLEICxA6j+hAmMzIlypy4xcBg1vKVnx89Wy0GbS+kf5cwCVFFzdCFh2XSCFNULS6
+       "license": "MIT",
+       "engines": {
+         "node": ">= 0.4"
+       }
+     },

```

```

+   "node_modules/es-object-atoms": {
+     "version": "1.1.2",
+     "resolved": "https://registry.npmjs.org/es-object-atoms/-/es-object-atoms-1.1.2.tgz",
+     "integrity": "sha512-HWcBoN6NileqtSydK2FqHbS/LoDd2pqrnQHLyJzBj4kOp/ky2MWMN694xOfkK8/SnUsW2DH7EfyVlydKCSn
+     "license": "MIT",
+     "dependencies": {
+       "es-errors": "^1.3.0"
+     },
+     "engines": {
+       "node": ">= 0.4"
+     }
+   },
+   "node_modules/es-set-tostringtag": {
+     "version": "2.1.0",
+     "resolved": "https://registry.npmjs.org/es-set-tostringtag/-/es-set-tostringtag-2.1.0.tgz",
+     "integrity": "sha512-j6vWzfrGVfyXxge+00x5sh6cvxAog0a/4Rdd2K36zCMV5eJ+/+tOAngRO8cODMNWbVRdVlmGZQL2YS3yR8k
+     "license": "MIT",
+     "dependencies": {
+       "es-errors": "^1.3.0",
+       "get-intrinsic": "^1.2.6",
+       "has-tostringtag": "^1.0.2",
+       "hasown": "^2.0.2"
+     },
+     "engines": {
+       "node": ">= 0.4"
+     }
+   },
+   "node_modules/ethereum-cryptography": {
+     "version": "2.2.1",
+     "resolved": "https://registry.npmjs.org/ethereum-cryptography/-/ethereum-cryptography-2.2.1.tgz",
+     "integrity": "sha512-r/W8lkHSiTlxUxW8Rf3u4HGB0xQweG2RyETjywyLkZSzLWoWAijRz8WCuOtJ6wah+avllXBqZuk29HCCvhE
+     "license": "MIT",
+     "dependencies": {
+       "@noble/curves": "1.4.2",
+       "@noble/hashes": "1.4.0",
+       "@scure/bip32": "1.4.0",
+       "@scure/bip39": "1.3.0"
+     }
+   },
+   "node_modules/ethereum-cryptography/node_modules/@noble/curves": {
+     "version": "1.4.2",

```

```

+     "resolved": "https://registry.npmjs.org/@noble/curves/-/curves-1.4.2.tgz",
+     "integrity": "sha512-
TavHr8qycMChk8UwMld0ZDRvatedkzWfH8IiaeGCfymOP5i0hSCozz9vHOL0nkwk7HRMlFnAiKpS2jrUmSy
+     "license": "MIT",
+     "dependencies": {
+       "@noble/curves": "1.4.0"
+     },
+     "funding": {
+       "url": "https://paulmillr.com/funding/"
+     }
+   },
+   "node_modules/ethereum-cryptography/node_modules/@noble/curves": {
+     "version": "1.4.0",
+     "resolved": "https://registry.npmjs.org/@noble/curves/-/curves-1.4.0.tgz",
+     "integrity": "sha512-
V1JJ1WTRUqHHrOSh597hURcMqVKVGL/ea3kv0gSnEdsEZ0/+VyPghM1lMNGc00z7CIQorSvbKpuJkxvuHbv
+     "license": "MIT",
+     "engines": {
+       "node": ">= 16"
+     },
+     "funding": {
+       "url": "https://paulmillr.com/funding/"
+     }
+   },
+   "node_modules/ethers": {
+     "version": "6.17.0",
+     "resolved": "https://registry.npmjs.org/ethers/-/ethers-6.17.0.tgz",
+     "integrity": "sha512-
BpyrpIPJ3ydEVow8zGaz1DuPS7YU8DcWxuBnY9a0UA/lvAPwrMr+EPXsfrul628SRaekPNeIM4UFh/91GWZ
+     "funding": [
+       {
+         "type": "individual",
+         "url": "https://github.com/sponsors/ethers-io/"
+       },
+       {
+         "type": "individual",
+         "url": "https://www.buymeacoffee.com/ricmoo"
+       }
+     ],
+     "license": "MIT",
+     "dependencies": {
+       "@adraffy/ens-normalize": "1.11.1",
+       "@noble/curves": "1.2.0",
+       "@noble/curves": "1.3.2",
+       "@types/node": "22.7.5",
+       "aes-js": "4.0.0-beta.5",
+       "tslib": "2.7.0",

```

```

+       "ws": "8.21.0"
+     },
+     "engines": {
+       "node": ">=14.0.0"
+     }
+   },
+   "node_modules/eventemitter3": {
+     "version": "5.0.1",
+     "resolved": "https://registry.npmjs.org/eventemitter3/-/eventemitter3-
5.0.1.tgz",
+     "integrity": "sha512-
GwkBvjISZK87ELrYOSYSUYeVic9mvLLf/nXalMOS5dYrgZq9o5OVkbZAVM06CVxYsCwH9BDZFP1QT1PA1j4
+     "license": "MIT"
+   },
+   "node_modules/follow-redirects": {
+     "version": "1.16.0",
+     "resolved": "https://registry.npmjs.org/follow-redirects/-/follow-
redirects-1.16.0.tgz",
+     "integrity": "sha512-
y5rN/uOsadFT/JfYwhxRS5R7Qce+g3zG97+JrtFZ1C9klX/W5hD7iiLzScI4nZqUS7DNUdhPgW4xI8W2LuX
+     "funding": [
+       {
+         "type": "individual",
+         "url": "https://github.com/sponsors/RubenVerborgh"
+       }
+     ],
+     "license": "MIT",
+     "engines": {
+       "node": ">=4.0"
+     },
+     "peerDependenciesMeta": {
+       "debug": {
+         "optional": true
+       }
+     }
+   },
+   "node_modules/form-data": {
+     "version": "4.0.6",
+     "resolved": "https://registry.npmjs.org/form-data/-/form-data-4.0.6.tgz",
+     "integrity": "sha512-
vKatAh4SlVfgbv+YtmhiRjhEMJsYpsG1Y2rMQtR+SVSbytsSD1YGzDIcrAJmdFec88u/+VoGmxnl+80gL1t
+     "license": "MIT",
+     "dependencies": {
+       "asynckit": "^0.4.0",
+       "combined-stream": "^1.0.8",
+       "es-set-tostringtag": "^2.1.0",
+       "hasown": "^2.0.4",

```



```
+ "mime-types": "^2.1.35"
+ },
+ "engines": {
+   "node": ">= 6"
+ }
+ },
+ "node_modules/function-bind": {
+   "version": "1.1.2",
+   "resolved": "https://registry.npmjs.org/function-bind/-/function-bind-1.1.2.tgz",
+   "integrity": "sha512-7XHNxH7qX9xG5mIwxkhumTox/MIRNcOgDrxWsMt2pAr23WHp6MrRlN7FBSFpCpr+oVO0F744iUgR82nJMfc9",
+   "license": "MIT",
+   "funding": {
+     "url": "https://github.com/sponsors/ljharb"
+   }
+ },
+ "node_modules/get-intrinsic": {
+   "version": "1.3.0",
+   "resolved": "https://registry.npmjs.org/get-intrinsic/-/get-intrinsic-1.3.0.tgz",
+   "integrity": "sha512-9fSjSaos/fRIVIp+xSJlE6lfwhES7LNtKaCBiamHsjr2na1BiABJPo0mOjjz8GJDURarmCPGgaiVg5mfjb9",
+   "license": "MIT",
+   "dependencies": {
+     "call-bind-apply-helpers": "^1.0.2",
+     "es-define-property": "^1.0.1",
+     "es-errors": "^1.3.0",
+     "es-object-atoms": "^1.1.1",
+     "function-bind": "^1.1.2",
+     "get-proto": "^1.0.1",
+     "gopd": "^1.2.0",
+     "has-symbols": "^1.1.0",
+     "hasown": "^2.0.2",
+     "math-intrinsics": "^1.1.0"
+   },
+   "engines": {
+     "node": ">= 0.4"
+   },
+   "funding": {
+     "url": "https://github.com/sponsors/ljharb"
+   }
+ },
+ "node_modules/get-proto": {
+   "version": "1.0.1",
+   "resolved": "https://registry.npmjs.org/get-proto/-/get-proto-1.0.1.tgz",
+   "integrity": "sha512-8ZyPvTQeA9DyXWw/1M8w70YsCD2jWkbRCsNWtJ8H/Ju2ARV0BOHviB8PSIHzYKIOghH8/4zdr/mffIQnA=="
```

```

sTSfBjoXBp89JvIKIefqw7U2CCebsc74kiY6awiGogKtoSGbgjYE/G/+l9sF3MWFPNc9IcoOC4ODfKHfxFn
+   "license": "MIT",
+   "dependencies": {
+     "dunder-proto": "^1.0.1",
+     "es-object-atoms": "^1.0.0"
+   },
+   "engines": {
+     "node": ">= 0.4"
+   }
+ },
+   "node_modules/google-protobuf": {
+     "version": "3.21.4",
+     "resolved": "https://registry.npmjs.org/google-protobuf/-/google-protobuf-
3.21.4.tgz",
+     "integrity": "sha512-
MnG7N936zcKTCO4Jd2PX2U96Kf9PxygAPKBUG+74LHzmHXmceN16MmRcdgZv+DGef/S9YvQAfRsNCn4cjf9
+     "license": "(BSD-3-Clause AND Apache-2.0)"
+   },
+   "node_modules/gopd": {
+     "version": "1.2.0",
+     "resolved": "https://registry.npmjs.org/gopd/-/gopd-1.2.0.tgz",
+     "integrity": "sha512-
ZUKRh6/kUFoAiTAtTYPZJ3hw9wNxx+BIBOIjnlG9PnrJsCcSjslwyYD6vJpaYtgzDrKYRSqf3006Rfa93
+     "license": "MIT",
+     "engines": {
+       "node": ">= 0.4"
+     },
+     "funding": {
+       "url": "https://github.com/sponsors/ljharb"
+     }
+   },
+   "node_modules/has-symbols": {
+     "version": "1.1.0",
+     "resolved": "https://registry.npmjs.org/has-symbols/-/has-symbols-
1.1.0.tgz",
+     "integrity": "sha512-
1cDNdwJ2Jaohmb3sg4OmKaMBwuC48sYni5HUw2DvsC8LjGTLK9h+eb1X6RyuOHe4hT0ULCW68iomhjUoKU
+     "license": "MIT",
+     "engines": {
+       "node": ">= 0.4"
+     },
+     "funding": {
+       "url": "https://github.com/sponsors/ljharb"
+     }
+   },
+   "node_modules/has-tostringtag": {
+     "version": "1.0.2",

```

```

+       "resolved": "https://registry.npmjs.org/has-tostringtag/-/has-tostringtag-
1.0.2.tgz",
+       "integrity": "sha512-
NqADB8VjPFLM2V0VvHUewwwsw0ZWBaIdgo+ieHtK3hasLz4qeCRjYcqfB6AQrBggRKppKF8L52/VqdVsO47
+       "license": "MIT",
+       "dependencies": {
+         "has-symbols": "^1.0.3"
+       },
+       "engines": {
+         "node": ">= 0.4"
+       },
+       "funding": {
+         "url": "https://github.com/sponsors/ljharb"
+       }
+     },
+     "node_modules/hasown": {
+       "version": "2.0.4",
+       "resolved": "https://registry.npmjs.org/hasown/-/hasown-2.0.4.tgz",
+       "integrity": "sha512-
T2UbfbBEF32wiepXIsmITW9+dDYC6wMh/t/vYA4tuOMKqWz/n3vr1NFSxQiyP+zk2mXsoMA/i/7qV6LKut1
+       "license": "MIT",
+       "dependencies": {
+         "function-bind": "^1.1.2"
+       },
+       "engines": {
+         "node": ">= 0.4"
+       }
+     },
+     "node_modules/https-proxy-agent": {
+       "version": "5.0.1",
+       "resolved": "https://registry.npmjs.org/https-proxy-agent/-/https-proxy-
agent-5.0.1.tgz",
+       "integrity": "sha512-
dFcAjPQTQFgoLMzC2VwU+C/CbS7uRL0lWmxDITmqm7C+7F0Odmj6s9l6alZc6AELXhrnggM2CeWSXHG0dX2Y
+       "license": "MIT",
+       "dependencies": {
+         "agent-base": "6",
+         "debug": "4"
+       },
+       "engines": {
+         "node": ">= 6"
+       }
+     },
+     "node_modules/math-intrinsics": {
+       "version": "1.1.0",
+       "resolved": "https://registry.npmjs.org/math-intrinsics/-/math-intrinsics-
1.1.0.tgz",

```

```

+     "integrity": "sha512-
/IXtbwEk5HTPyEwyKX6hGkYXxM9nbj64B+ilVJnC/R6B0pH5G4V3b0pVbL7DBj4tkhBAppbQUlf6F6Xl9Lf
+     "license": "MIT",
+     "engines": {
+       "node": ">= 0.4"
+     }
+   },
+   "node_modules/mime-db": {
+     "version": "1.52.0",
+     "resolved": "https://registry.npmjs.org/mime-db/-/mime-db-1.52.0.tgz",
+     "integrity": "sha512-
sPU4uV7dYlvtWJxwwxHD0PuihVNiE7TyAbQ5SWxDCB9mUYvOgroQQOwYQQOKPJ8CIbE+1ETVlOoK1UC2nU3c
+     "license": "MIT",
+     "engines": {
+       "node": ">= 0.6"
+     }
+   },
+   "node_modules/mime-types": {
+     "version": "2.1.35",
+     "resolved": "https://registry.npmjs.org/mime-types/-/mime-types-
2.1.35.tgz",
+     "integrity": "sha512-
ZDY+bPm5zTTF+YpCrAU9nK0UgICYPT0QtT1NZWFv4s++TNkcgVaT0g6+4R2uI4MjQjzysHBlzXuWL50hzae
+     "license": "MIT",
+     "dependencies": {
+       "mime-db": "1.52.0"
+     },
+     "engines": {
+       "node": ">= 0.6"
+     }
+   },
+   "node_modules/ms": {
+     "version": "2.1.3",
+     "resolved": "https://registry.npmjs.org/ms/-/ms-2.1.3.tgz",
+     "integrity": "sha512-
6FlzubTLZG3J2a/NVCAleEhjzq5oxgHyaCU9yYXvcLsvoVaHJq/s5xXI6/XP6tz7R9xAOtHnSO/tXtF3WF
+     "license": "MIT"
+   },
+   "node_modules/proxy-from-env": {
+     "version": "2.1.0",
+     "resolved": "https://registry.npmjs.org/proxy-from-env/-/proxy-from-env-
2.1.0.tgz",
+     "integrity": "sha512-
cJ+oHTWlVAEa8cJslgmUZrc+sJRKgaKl3Zy6+PV38hZe/V6Z14TbCuXcan9F9ghlz4QrFr2c92TNF82Uk
+     "license": "MIT",
+     "engines": {
+       "node": ">=10"
+     }
+   }

```

```

+     }
+   },
+   "node_modules/semver": {
+     "version": "7.7.1",
+     "resolved": "https://registry.npmjs.org/semver/-/semver-7.7.1.tgz",
+     "integrity": "sha512-hlq8tAfn0m/6lp4BVRcPzIGr6LKiMwo4VM6dGi6pt4qcRkmNzTcWq6eCEjEh+qXjkMDvPlOFFSGwQjoEa6c
hlq8tAfn0m/6lp4BVRcPzIGr6LKiMwo4VM6dGi6pt4qcRkmNzTcWq6eCEjEh+qXjkMDvPlOFFSGwQjoEa6c
+     "license": "ISC",
+     "bin": {
+       "semver": "bin/semver.js"
+     },
+     "engines": {
+       "node": ">=10"
+     }
+   },
+   "node_modules/tronweb": {
+     "version": "6.5.0",
+     "resolved": "https://registry.npmjs.org/tronweb/-/tronweb-6.5.0.tgz",
+     "integrity": "sha512-
10lkUqrFZpVSx0Gvwi9311iOVHbALobHg4SEEvOR3rhAUy+IjUIBCCwJB/u8QFpxx9GK6R6Njyqx45BMX3t
+     "license": "MIT",
+     "dependencies": {
+       "axios": "1.18.0",
+       "bignumber.js": "9.1.2",
+       "ethereum-cryptography": "2.2.1",
+       "ethers": "6.17.0",
+       "eventemitter3": "5.0.1",
+       "google-protobuf": "3.21.4",
+       "semver": "7.7.1",
+       "tslib": "2.7.0",
+       "validator": "13.15.23"
+     }
+   },
+   "node_modules/tslib": {
+     "version": "2.7.0",
+     "resolved": "https://registry.npmjs.org/tslib/-/tslib-2.7.0.tgz",
+     "integrity": "sha512-
gLXCKdN1/j47AiHiOkJN69hJmcbGTHi0ImLmbYLHykhgeN0jVGola9yVjFgzCUklsZQMw55o+dW7IXv3RCx
+     "license": "0BSD"
+   },
+   "node_modules/undici-types": {
+     "version": "6.19.8",
+     "resolved": "https://registry.npmjs.org/undici-types/-/undici-types-
6.19.8.tgz",
+     "integrity": "sha512-
ve2KP6f/JnbPBFyobGHuerC9g1FYGn/F8n1LWTWnxCezd6IfqTwUQcNXgEtmMQ6DlRrClhrSrBnCZPokRrI
+     "license": "MIT"

```

```

+   },
+   "node_modules/validator": {
+     "version": "13.15.23",
+     "resolved": "https://registry.npmjs.org/validator/-/validator-
13.15.23.tgz",
+     "integrity": "sha512-
4yoz1kEWqUjzi5zsPbAS/903QXSYp0UOtHsPpp7p9rHAW/W+dkInskAE386Fat3oKRROwO98d9ZB0G4cObc
+     "license": "MIT",
+     "engines": {
+       "node": ">= 0.10"
+     }
+   },
+   "node_modules/ws": {
+     "version": "8.21.0",
+     "resolved": "https://registry.npmjs.org/ws/-/ws-8.21.0.tgz",
+     "integrity": "sha512-
Vsp28b7DRcimFQvrqu2Wek3zliYxDCWqHYB8Qsnk/S4RfaCQzPGPyBNuVjJV3cd6UiKtUtp6sNM77gWvzcc
+     "license": "MIT",
+     "engines": {
+       "node": ">=10.0.0"
+     },
+     "peerDependencies": {
+       "bufferutil": "^4.0.1",
+       "utf-8-validate": ">=5.0.2"
+     },
+     "peerDependenciesMeta": {
+       "bufferutil": {
+         "optional": true
+       },
+       "utf-8-validate": {
+         "optional": true
+       }
+     }
+   }
+ }
+ }
+ }

```

```
diff --git a/contracts/tron/tronbox/package.json
```

```
b/contracts/tron/tronbox/package.json
```

```
index c53ef06..7f1b535 100644
```

```
--- a/contracts/tron/tronbox/package.json
```

```
+++ b/contracts/tron/tronbox/package.json
```

```
@@ -4,6 +4,8 @@
```

```
    "version": "0.2.0",
```

```
    "description": "TronBox project for D3R-AC's TRON contracts. Nested here, one
level below ../package.json, deliberately: TronBox's config (tronbox-config.js)
must stay CommonJS, while ../package.json is 'type': 'module' (required
unconditionally by Hardhat 3, which lives at that level instead, since Hardhat 3
```

requires its sources inside its own project directory). Node resolves module type from the nearest package.json, so this directory boundary keeps the two tools' conflicting requirements from colliding. contracts_directory in tronbox-config.js points to '../contracts' -- the same .sol sources Hardhat tests against, so there's one source of truth for both tools.",

```
    "dependencies": {  
-     "dotenv": "^16.4.5"  
+     "dotenv": "^16.4.5",  
+     "ethers": "^6.17.0",  
+     "tronweb": "^6.5.0"  
    }  
  }  
}
```

```
diff --git a/contracts/tron/tronbox/scripts/check-deploy-balance.js  
b/contracts/tron/tronbox/scripts/check-deploy-balance.js  
new file mode 100644  
index 0000000..4190206
```

```
--- /dev/null  
+++ b/contracts/tron/tronbox/scripts/check-deploy-balance.js  
@@ -0,0 +1,86 @@  
+#!/usr/bin/env node  
+// Pre-flight check run by .github/workflows/deploy-testnet.yml before  
+// `tronbox migrate`. Fails fast with a clear faucet link if the deploy  
+// wallet has no testnet TRX, instead of letting the migration die  
+// partway through -- deploying 7 contracts + wiring costs real (though  
+// free-to-obtain) gas, and a mid-migration out-of-gas failure is much  
+// more annoying to diagnose and recover from than this one check.  
+//  
+// Reads the private key from the same env vars tronbox-config.js  
+// does (TRON_PRIVATE_KEY_SHASTA / TRON_PRIVATE_KEY_NILE), derives the  
+// corresponding address locally (never sent anywhere), and queries  
+// that address's public balance via the network's own TronGrid  
+// endpoint -- no separate credentials needed for this step, since  
+// balance lookups are a public read.  
+const { TronWeb } = require("tronweb");  
+  
+const NETWORKS = {  
+  shasta: {  
+    fullHost: "https://api.shasta.trongrid.io",  
+    keyEnv: "TRON_PRIVATE_KEY_SHASTA",  
+    faucet: "https://www.trongrid.io/shasta",  
+  },  
+  nile: {  
+    fullHost: "https://nile.trongrid.io",  
+    keyEnv: "TRON_PRIVATE_KEY_NILE",  
+    faucet: "https://nileex.io/join/getJoinPage",  
+  },  
+};
```

```

+
+// Rough floor: 7 contract deployments + ~15 wiring calls on Shasta/Nile
+// have run well under 2000 TRX in practice for this suite at the
+// current optimizer settings, but testnet fee markets drift -- this is
+// a "did you forget to fund it at all" check, not a precise estimate.
+// Bump it if a real run gets partway through and still runs out.
+const MIN_BALANCE_TRX = 500;
+
+async function main() {
+  const network = process.argv[2];
+  const config = NETWORKS[network];
+  if (!config) {
+    console.error(`check-deploy-balance: unknown network '${network}'. Expected
one of: ${Object.keys(NETWORKS).join(", ")}`);
+    process.exit(1);
+  }
+
+  const privateKey = process.env[config.keyEnv];
+  if (!privateKey) {
+    console.error(`check-deploy-balance: ${config.keyEnv} is not set. Add it
under this repo's`);
+    console.error("Settings -> Secrets and variables -> Actions before running
this workflow.");
+    process.exit(1);
+  }
+
+  const tronWeb = new TronWeb({ fullHost: config.fullHost, privateKey });
+  const address = tronWeb.address.fromPrivateKey(privateKey);
+
+  let sunBalance;
+  try {
+    sunBalance = await tronWeb.trx.getBalance(address);
+  } catch (err) {
+    console.error(`check-deploy-balance: couldn't reach ${config.fullHost} to
check balance: ${err.message}`);
+    console.error("This usually means TronGrid is temporarily unreachable --
retry the workflow run.");
+    process.exit(1);
+  }
+
+  const trxBalance = sunBalance / 1_000_000; // 1 TRX = 1,000,000 sun
+
+  console.log(`Deploy address (${network}): ${address}`);
+  console.log(`Current balance: ${trxBalance} TRX`);
+
+  if (trxBalance < MIN_BALANCE_TRX) {
+    console.error("");

```



```

+   console.error(`check-deploy-balance: balance is below the ${MIN_BALANCE_TRX}
TRX floor for a full`);
+   console.error("deployment + Hub wiring + MultiSigAdmin submission. Fund this
exact address");
+   console.error(`${address}) from the ${network} faucet before re-running:`);
+   console.error(` ${config.faucet}`);
+   console.error("");
+   console.error("Nothing has been sent yet -- this check runs before any
transaction.");
+   process.exit(1);
+ }
+
+ console.log("Balance check passed -- proceeding to migration.");
+}
+
+main().catch((err) => {
+   console.error("check-deploy-balance: unexpected error:", err);
+   process.exit(1);
+});
diff --git a/contracts/tron/tronbox/scripts/find-vanity-create2-salt.js
b/contracts/tron/tronbox/scripts/find-vanity-create2-salt.js
new file mode 100644
index 00000000..f9b6254
--- /dev/null
+++ b/contracts/tron/tronbox/scripts/find-vanity-create2-salt.js
@@ -0,0 +1,157 @@
+#!/usr/bin/env node
+// Finds a `salt` for D3RACTokenFactory.deployToken(...) that makes the
+// resulting D3RACToken contract address contain a chosen substring
+// (e.g. "D3RAC") - D3RAC's on-chain identity, made to visibly read as
+// D3RAC's own instead of landing wherever a plain sequential
+// deployment happens to put it.
+//
+// ZERO private key material is used or needed anywhere in this
+// script. A CREATE2 address is a pure function of three PUBLIC
+// values (deployer/factory address, salt, keccak256(init code)) - see
+// D3RACTokenFactory.sol's own doc comment for the full explanation.
+// This grinds candidate salts and checks the resulting PUBLIC address
+// against the target substring; nothing here can sign or send a
+// transaction. Once a salt is found, a real transaction calling
+// `factory.deployToken(salt, initialSupply, owner)` is what actually
+// creates the contract there - that step needs a real signer, same as
+// any other deployment, and is out of scope for this script on purpose.
+//
+// IMPORTANT - which artifact to point this at:
+// The init code hash includes the compiler's own metadata bytes
+// (source hash + compiler version), which differ between ordinary

```

```

+// solc and TRON's own solc-bin fork even for identical source and
+// version number. A salt found against the wrong compiler's output
+// will NOT produce the address you expect on real TRON. Always run
+// this against the JSON artifact from `tronbox compile` (default
+// path below), not Hardhat's `artifacts/` - Hardhat's build is useful
+// only for locally verifying the grinding logic itself (see
+// test/D3RACTokenFactory.test.mjs), not for a real target address.
+//
+// Usage:
+//   node scripts/find-vanity-create2-salt.js \
+//     --factory TFactoryAddressGoesHere... \
+//     --owner TOwnerAddressGoesHere... \
+//     --initial-supply 1000000 \
+//     --contains D3RAC \
+//     [--ignore-case] [--max-attempts 5000000] \
+//     [--artifact ./build/contracts/D3RACToken.json]
+const fs = require("fs");
+const path = require("path");
+const crypto = require("crypto");
+const { ethers } = require("ethers");
+const { TronWeb } = require("tronweb");
+
+function parseArgs(argv) {
+  const args = {};
+  for (let i = 2; i < argv.length; i++) {
+    if (argv[i].startsWith("--")) {
+      const key = argv[i].slice(2);
+      const next = argv[i + 1];
+      if (next === undefined || next.startsWith("--")) {
+        args[key] = true;
+      } else {
+        args[key] = next;
+        i++;
+      }
+    }
+  }
+  return args;
+}
+
+function tronToEvmAddress(tronBase58) {
+  const hex = TronWeb.address.toHex(tronBase58); // "41" + 40 hex chars
+  if (!hex.startsWith("41") || hex.length !== 42) {
+    throw new Error(`Unexpected TRON hex address format for ${tronBase58}:
+${hex}`);
+  }
+  return "0x" + hex.slice(2);
+}

```

```

+
+function evmAddressToTron(evmAddress) {
+  const hex = "41" + evmAddress.replace(/^0x/, "");
+  return TronWeb.address.fromHex(hex);
+}
+
+async function main() {
+  const args = parseArgs(process.argv);
+
+  const required = ["factory", "owner", "contains"];
+  for (const key of required) {
+    if (!args[key]) {
+      console.error(`Missing required --${key}. See this script's header comment
for usage.`);
+      process.exit(1);
+    }
+  }
+
+  const artifactPath = args.artifact
+    ? path.resolve(args.artifact)
+    : path.resolve(__dirname, "../build/contracts/D3RACToken.json");
+
+  if (!fs.existsSync(artifactPath)) {
+    console.error(`Artifact not found at ${artifactPath}.`);
+    console.error("Run `tronbox compile` first (see this script's header comment
for why it");
+    console.error("must be TronBox's own build, not Hardhat's), or pass --
artifact explicitly.");
+    process.exit(1);
+  }
+
+  const artifact = JSON.parse(fs.readFileSync(artifactPath, "utf8"));
+  const bytecode = artifact.bytecode ?? artifact.data?.bytecode?.object;
+  if (!bytecode) {
+    console.error(`Couldn't find a "bytecode" field in ${artifactPath} -
unexpected artifact shape.`);
+    process.exit(1);
+  }
+
+  const initialSupply = BigInt(args["initial-supply"] ?? "0");
+  const ownerEvmAddress = tronToEvmAddress(args.owner);
+  const factoryEvmAddress = tronToEvmAddress(args.factory);
+
+  const abiCoder = ethers.AbiCoder.defaultAbiCoder();
+  const encodedArgs = abiCoder.encode(["uint256", "address"], [initialSupply,
ownerEvmAddress]);
+  const initCode = ethers.concat([bytecode.startsWith("0x") ? bytecode : "0x" +

```

```

bytecode, encodedArgs]);
+   const initCodeHash = ethers.keccak256(initCode);
+
+   const target = args["ignore-case"] ? String(args.contains).toLowerCase() :
String(args.contains);
+   const maxAttempts = Number(args["max-attempts"] ?? 2_000_000);
+
+   console.log(`Factory (TRON):  ${args.factory}`);
+   console.log(`Factory (EVM):   ${factoryEvmAddress}`);
+   console.log(`Owner (TRON):    ${args.owner}`);
+   console.log(`Init code hash:  ${initCodeHash}`);
+   console.log(`Searching for "${args.contains}" in the resulting TRON address
(case-${args["ignore-case"] ? "insensitive" : "sensitive"})...`);
+   console.log("");
+
+   const startedAt = Date.now();
+   for (let attempt = 0; attempt < maxAttempts; attempt++) {
+     const salt = "0x" + crypto.randomBytes(32).toString("hex");
+     const predictedEvm = ethers.create2Address(factoryEvmAddress, salt,
initCodeHash);
+     const predictedTron = evmAddressToTron(predictedEvm);
+
+     const haystack = args["ignore-case"] ? predictedTron.toLowerCase() :
predictedTron;
+     if (haystack.includes(target)) {
+       const elapsedSec = ((Date.now() - startedAt) / 1000).toFixed(1);
+       console.log(`Match found after ${attempt + 1} attempts (${elapsedSec}s):`);
+       console.log(`  salt:           ${salt}`);
+       console.log(`  predicted (TRON): ${predictedTron}`);
+       console.log(`  predicted (EVM):  ${predictedEvm}`);
+       console.log("");
+       console.log("Next step: call factory.deployToken(salt, initialSupply,
owner) with a real");
+       console.log("signer, using the exact same initialSupply/owner used above.
The deployed");
+       console.log("token's address will be exactly the one predicted here.");
+       return;
+     }
+
+     if (attempt > 0 && attempt % 200_000 === 0) {
+       const elapsedSec = ((Date.now() - startedAt) / 1000).toFixed(1);
+       console.log(`...${attempt} attempts checked (${elapsedSec}s), still
searching`);
+     }
+   }
+
+   console.error(`No match found in ${maxAttempts} attempts. A longer/rarer

```

```

substring needs`);
+ console.error("exponentially more attempts (TRON addresses are base58: ~58x
harder per extra");
+ console.error("character) -- try a shorter substring, --ignore-case, or a
higher --max-attempts.");
+ process.exit(1);
+}
+
+main().catch((err) => {
+ console.error("find-vanity-create2-salt: unexpected error:", err);
+ process.exit(1);
+});
diff --git a/contracts/tron/tronbox/scripts/find-vanity-tron-wallet.js
b/contracts/tron/tronbox/scripts/find-vanity-tron-wallet.js
new file mode 100644
index 0000000..a607cd4
--- /dev/null
+++ b/contracts/tron/tronbox/scripts/find-vanity-tron-wallet.js
@@ -0,0 +1,119 @@
+#!/usr/bin/env node
+// Generates fresh TRON keypairs locally (never sent anywhere -- see
+// below) and searches for one whose address contains a chosen
+// substring, e.g. "D3RAC".
+//
+// READ THIS BEFORE RUNNING IT ANYWHERE THAT MATTERS:
+//
+// Unlike D3RACTokenFactory's CREATE2 contract-address grind (pure
+// public math, no key material involved at all), a wallet address
+// only exists once a real private key has been generated -- there is
+// no way to search for a vanity EOA address without creating one real
+// keypair per attempt. That means whoever RUNS this script ends up
+// holding a genuine private key the moment it finds a match.
+//
+// Consequences that follow directly from that:
+// - Run this on a machine you trust, offline if possible. It makes
+//   zero network calls (createAccount() is pure local
+//   secp256k1 math -- verified: no fullHost is ever contacted), but
+//   "no network calls" doesn't help if the machine itself is
+//   compromised or logs stdout somewhere you don't control.
+// - The found private key is printed to stdout. Treat that terminal
+//   output, your shell history, and any log capture as sensitive
+//   the instant this prints -- move the key into a real secret
+//   store (a hardware wallet, TronLink, or this repo's GitHub
+//   Actions secrets per docs/deployment-guide.md) and don't leave
+//   it sitting in a terminal scrollbar.
+// - If you ever paste this script's output into a chat, a shared
+//   document, a GitHub issue, a support ticket -- anywhere another

```

```

+//      party (including an AI assistant) could see it -- that key is
+//      burned. Generate a new one. There is no way to "undo" a private
+//      key having been seen by someone else.
+//      - This script cannot run on your behalf from inside a shared or
+//      third-party-operated environment for anything you intend to
+//      hold real value -- that is precisely the scenario the warnings
+//      above are about, not a hypothetical.
+//
+// Usage:
+//   node scripts/find-vanity-tron-wallet.js --contains D3RAC [--ignore-case] [--
max-attempts 50000000]
+//
+// Performance note: unlike CREATE2 grinding (a single keccak256 per
+// attempt), each attempt here does a real secp256k1 keypair
+// generation -- meaningfully slower per attempt. Expect roughly an
+// order of magnitude fewer attempts/sec than
+// find-vanity-create2-salt.js on the same hardware; run
+// --contains D3R (3 chars) first to see this machine's real
+// attempts/sec before committing to a full 5-character "D3RAC" run.
+
+const { TronWeb } = require("tronweb");
+
+function parseArgs(argv) {
+  const args = {};
+  for (let i = 2; i < argv.length; i++) {
+    if (argv[i].startsWith("--")) {
+      const key = argv[i].slice(2);
+      const next = argv[i + 1];
+      if (next === undefined || next.startsWith("--")) {
+        args[key] = true;
+      } else {
+        args[key] = next;
+        i++;
+      }
+    }
+  }
+  return args;
+}
+
+async function main() {
+  const args = parseArgs(process.argv);
+  if (!args.contains) {
+    console.error("Missing required --contains <substring>. See this script's
header for usage.");
+    process.exit(1);
+  }
+}
+

```

```

+ const target = args["ignore-case"] ? String(args.contains).toLowerCase() :
String(args.contains);
+ const maxAttempts = Number(args["max-attempts"] ?? 20_000_000);
+
+ const tronWeb = new TronWeb({ fullHost: "https://api.shasta.trongrid.io" }); //
fullHost is required by the constructor but createAccount() below never contacts
it
+
+ console.log(`Searching for "${args.contains}" in a freshly generated TRON
address`);
+ console.log(`(case-${args["ignore-case"]} ? "insensitive" : "sensitive"). Every
attempt generates a`);
+ console.log("real keypair locally -- no network calls. See this script's header
comment");
+ console.log("for what to do (and not do) with whatever key this finds.\n");
+
+ const startedAt = Date.now();
+ for (let attempt = 0; attempt < maxAttempts; attempt++) {
+   const account = await tronWeb.createAccount();
+   const address = account.address.base58;
+   const haystack = args["ignore-case"] ? address.toLowerCase() : address;
+
+   if (haystack.includes(target)) {
+     const elapsedSec = ((Date.now() - startedAt) / 1000).toFixed(1);
+     console.log(`Match found after ${attempt + 1} attempts (${elapsedSec}s):`);
+     console.log(`  address:      ${address}`);
+     console.log(`  private key: ${account.privateKey}`);
+     console.log("");
+     console.log("This key exists only in this process's memory and stdout right
now --");
+     console.log("move it into real custody (hardware wallet / TronLink / GitHub
Actions");
+     console.log("secret) immediately, and treat this terminal's scrollback as
sensitive");
+     console.log("until you've done so.");
+     return;
+   }
+
+   if (attempt > 0 && attempt % 20_000 === 0) {
+     const elapsedSec = ((Date.now() - startedAt) / 1000).toFixed(1);
+     const rate = Math.round(attempt / (elapsedSec || 1));
+     console.log(`...${attempt} attempts checked (${elapsedSec}s, ~${rate}/s),
still searching`);
+   }
+ }
+
+ console.error(`No match found in ${maxAttempts} attempts. Try a shorter

```

```

substring, --ignore-case,`);
+ console.error("or a higher --max-attempts -- run --contains D3R first to gauge
this machine's");
+ console.error("real attempts/sec before committing to the full 5-character
\"D3RAC\".");
+ process.exit(1);
+}
+
+main().catch((err) => {
+ console.error("find-vanity-tron-wallet: unexpected error:", err);
+ process.exit(1);
+});
diff --git a/frontend/src/App.tsx b/frontend/src/App.tsx
index 9d8belf..f7f5546 100644
--- a/frontend/src/App.tsx
+++ b/frontend/src/App.tsx
@@ -5,6 +5,7 @@ import NavBar from "../components/NavBar";
import Footer from "../components/Footer";
import Landing from "../pages/Landing";
import Dashboard from "../pages/Dashboard";
+import Wallet from "../pages/Wallet";
import Disburse from "../pages/Disburse";
import NotFound from "../pages/NotFound";

@@ -19,6 +20,7 @@ export default function App() {
  <Routes>
    <Route path="/" element={<Landing />} />
    <Route path="/dashboard" element={<Dashboard />} />
+    <Route path="/wallet" element={<Wallet />} />
    <Route path="/disburse" element={<Disburse />} />
    <Route path="*" element={<NotFound />} />
  </Routes>
diff --git a/frontend/src/components/FormField.tsx
b/frontend/src/components/FormField.tsx
new file mode 100644
index 0000000..3c56c76
--- /dev/null
+++ b/frontend/src/components/FormField.tsx
@@ -0,0 +1,20 @@
+import type { CSSProperties, ReactNode } from "react";
+
+export function Field({ label, children }: { label: string; children: ReactNode
}) {
+  return (
+    <label style={{ display: "flex", flexDirection: "column", gap: 6 }}>
+      <span style={{ fontSize: 13, color: "var(--text-muted)" }}>{label}</span>
+      {children}
+    </label>
+  );
}

```



```

+   </label>
+ );
+}
+
+export const inputStyle: CSSProperties = {
+  background: "var(--bg-raised)",
+  border: "1px solid var(--border)",
+  borderRadius: 8,
+  padding: "10px 12px",
+  color: "var(--text)",
+  fontFamily: "var(--font-mono)",
+  fontSize: 14,
+};
diff --git a/frontend/src/components/NavBar.tsx
b/frontend/src/components/NavBar.tsx
index 3639774..c8dcb1d 100644
--- a/frontend/src/components/NavBar.tsx
+++ b/frontend/src/components/NavBar.tsx
@@ -4,6 +4,7 @@ import ChainSelector from "../ChainSelector";
const LINKS = [
  { to: "/", label: "Overview", end: true },
  { to: "/dashboard", label: "Risk Dashboard", end: false },
+ { to: "/wallet", label: "Wallet", end: false },
  { to: "/disburse", label: "Disbursement", end: false },
];

diff --git a/frontend/src/lib/casperAdapter.ts b/frontend/src/lib/casperAdapter.ts
index b895598..be0e7e2 100644
--- a/frontend/src/lib/casperAdapter.ts
+++ b/frontend/src/lib/casperAdapter.ts
@@ -36,6 +36,11 @@ class CasperAdapter implements ChainAdapter {
  return this.address;
}

+ async getNativeBalance(): Promise<string> {
+   throw new Error("Casper balance reading is not available yet – Casper
contracts are in progress.");
+ }
+
+
  async getTokenBalance(_tokenContract: string): Promise<TokenBalance> {
    throw new Error("Casper token contracts are not deployed yet – see project
README status.");
  }
diff --git a/frontend/src/lib/chainAdapter.ts b/frontend/src/lib/chainAdapter.ts
index 93ea9e7..41d38c1 100644
--- a/frontend/src/lib/chainAdapter.ts

```

```

+++ b/frontend/src/lib/chainAdapter.ts
@@ -23,6 +23,7 @@ export interface ChainAdapter {
    isWalletAvailable(): boolean;
    connect(): Promise<string>; // returns connected address
    getAddress(): string | null;
+   getNativeBalance(): Promise<string>; // human-readable native token (TRX/CSPR)
balance of the connected address
    getTokenBalance(tokenContract: string): Promise<TokenBalance>;
    disburse(params: {
        tokenContract: string;
diff --git a/frontend/src/lib/contractAddresses.ts
b/frontend/src/lib/contractAddresses.ts
new file mode 100644
index 0000000..9f18ebd
--- /dev/null
+++ b/frontend/src/lib/contractAddresses.ts
@@ -0,0 +1,51 @@
+/**
+ * Deployed contract addresses, by network.
+ *
+ * This file is deliberately empty of real addresses right now – no
+ * D3R·AC contract has been deployed to any network yet (see
+ * contracts/tron/README.md's "Known limitations" and
+ * docs/deployment-guide.md). Fill in a network's block the moment
+ * `.github/workflows/deploy-testnet.yml` finishes a real run: that
+ * workflow's log prints the exact values that go here, under
+ * "=== D3R-AC deployment complete ===".
+ *
+ * Never hand-write a placeholder-looking address into this file "to
+ * make the UI look finished" – every field the UI reads from here
+ * either is a real, currently-live contract, or is `null` and the UI
+ * shows an honest "not deployed on this network yet" state (see
+ * Wallet.tsx). A fake-but-plausible address here is worse than an
+ * empty one: someone could paste it into a real wallet and send real
+ * funds into the void.
+ */
+
+export interface D3racDeployment {
+  token: string | null; // D3RACToken (the D3RAC ERC/TRC-style token itself)
+  hub: string | null; // D3RACHub (the single admin-facing entry point)
+}
+
+export const CONTRACT_ADDRESSES: Record<"tron" | "casper", Record<string,
D3racDeployment>> = {
+  tron: {
+    shasta: { token: null, hub: null },
+    nile: { token: null, hub: null },

```

```

+   mainnet: { token: null, hub: null },
+ },
+ casper: {
+   testnet: { token: null, hub: null },
+   mainnet: { token: null, hub: null },
+ },
+};
+
+/** Reads VITE_TRON_NETWORK (see frontend/.env.example) with a safe default. */
+export function currentTronNetwork(): "shasta" | "nile" | "mainnet" {
+  const v = import.meta.env.VITE_TRON_NETWORK;
+  return v === "nile" || v === "mainnet" ? v : "shasta";
+}
+
+/** The D3RACToken address for the given chain's currently-configured network, or
+null if not deployed there yet. */
+export function currentTokenAddress(chainId: "tron" | "casper"): string | null {
+  if (chainId === "tron") {
+    return CONTRACT_ADDRESSES.tron[currentTronNetwork()]?.token ?? null;
+  }
+  // Casper network selection isn't wired up yet - see casperAdapter.ts.
+  return CONTRACT_ADDRESSES.casper.testnet?.token ?? null;
+}
diff --git a/frontend/src/lib/tronAdapter.ts b/frontend/src/lib/tronAdapter.ts
index 34a610d..9df022f 100644
--- a/frontend/src/lib/tronAdapter.ts
+++ b/frontend/src/lib/tronAdapter.ts
@@ -41,6 +41,14 @@ class TronAdapter implements ChainAdapter {
   return this.address;
 }

+  async getNativeBalance(): Promise<string> {
+    if (!this.address) {
+      throw new Error("Wallet not connected.");
+    }
+    const sun = await window.tronWeb.trx.getBalance(this.address);
+    return formatUnits(BigInt(sun), 6); // 1 TRX = 10^6 sun
+  }
+
   async getTokenBalance(tokenContract: string): Promise<TokenBalance> {
     if (!this.address) {
       throw new Error("Wallet not connected.");
diff --git a/frontend/src/pages/Disburse.tsx b/frontend/src/pages/Disburse.tsx
index 2586d67..2b20479 100644
--- a/frontend/src/pages/Disburse.tsx
+++ b/frontend/src/pages/Disburse.tsx
@@ -2,6 +2,7 @@ import { useState } from "react";

```

```

import type { FormEvent } from "react";
import { useWallet } from "../context/useWallet";
import type { DisbursementResult, TokenBalance } from "../lib/chainAdapter";
+import { Field, inputStyle } from "../components/FormField";

export default function Disburse() {
  const { adapter, chainId, address, connect, connecting } = useWallet();
@@ -110,21 +111,3 @@ export default function Disburse() {
  );
}

-function Field({ label, children }: { label: string; children: React.ReactNode })
{
-  return (
-    <label style={{ display: "flex", flexDirection: "column", gap: 6 }}>
-      <span style={{ fontSize: 13, color: "var(--text-muted)" }}>{label}</span>
-      {children}
-    </label>
-  );
-}
-
-const inputStyle: React.CSSProperties = {
-  background: "var(--bg-raised)",
-  border: "1px solid var(--border)",
-  borderRadius: 8,
-  padding: "10px 12px",
-  color: "var(--text)",
-  fontFamily: "var(--font-mono)",
-  fontSize: 14,
-};
diff --git a/frontend/src/pages/Wallet.tsx b/frontend/src/pages/Wallet.tsx
new file mode 100644
index 00000000..feb2e79
--- /dev/null
+++ b/frontend/src/pages/Wallet.tsx
@@ -0,0 +1,209 @@
+import { useEffect, useState } from "react";
+import type { FormEvent } from "react";
+import { useWallet } from "../context/useWallet";
+import type { DisbursementResult, TokenBalance } from "../lib/chainAdapter";
+import { currentTokenAddress, currentTronNetwork } from
+ "../lib/contractAddresses";
+import { DONATION_ADDRESSES } from "../lib/donationAddresses";
+import { Field, inputStyle } from "../components/FormField";
+
+export default function Wallet() {
+  const { adapter, chainId, address, connect, connecting } = useWallet();

```

```

+   const tokenAddress = currentTokenAddress(chainId);
+
+   const [nativeBalance, setNativeBalance] = useState<string | null>(null);
+   const [tokenBalance, setTokenBalance] = useState<TokenBalance | null>(null);
+   const [loadingBalances, setLoadingBalances] = useState(false);
+   const [balanceErr, setBalanceErr] = useState<string | null>(null);
+
+   const [to, setTo] = useState("");
+   const [amount, setAmount] = useState("");
+   const [sendResult, setSendResult] = useState<DisbursementResult | null>(null);
+   const [sendErr, setSendErr] = useState<string | null>(null);
+   const [sending, setSending] = useState(false);
+
+   const [copied, setCopied] = useState(false);
+
+   useEffect(() => {
+     if (!address) return;
+     let cancelled = false;
+     setLoadingBalances(true);
+     setBalanceErr(null);
+
+     (async () => {
+       try {
+         const native = await adapter.getNativeBalance();
+         if (!cancelled) setNativeBalance(native);
+       } catch (e) {
+         if (!cancelled) setBalanceErr(e instanceof Error ? e.message : "Could not
not read balance.");
+       }
+
+       if (tokenAddress) {
+         try {
+           const tok = await adapter.getTokenBalance(tokenAddress);
+           if (!cancelled) setTokenBalance(tok);
+         } catch (e) {
+           if (!cancelled) setBalanceErr(e instanceof Error ? e.message : "Could
not read token balance.");
+         }
+       }
+
+       if (!cancelled) setLoadingBalances(false);
+     })();
+
+     return () => {
+       cancelled = true;
+     };
+     // eslint-disable-next-line react-hooks/exhaustive-deps

```

```

+ }, [address, chainId, tokenAddress]);
+
+ async function copyAddress() {
+   if (!address) return;
+   await navigator.clipboard.writeText(address);
+   setCopied(true);
+   setTimeout(() => setCopied(false), 1500);
+ }
+
+ async function handleSend(e: FormEvent) {
+   e.preventDefault();
+   if (!tokenAddress) return;
+   setSendErr(null);
+   setSendResult(null);
+   setSending(true);
+   try {
+     const result = await adapter.disburse({ tokenContract: tokenAddress, to,
amount });
+     setSendResult(result);
+     setTo("");
+     setAmount("");
+     // Balance just changed on-chain – reflect it without waiting for a manual
refresh.
+     const [native, tok] = await Promise.all([adapter.getNativeBalance(),
adapter.getTokenBalance(tokenAddress)]);
+     setNativeBalance(native);
+     setTokenBalance(tok);
+   } catch (e) {
+     setSendErr(e instanceof Error ? e.message : "Send failed.");
+   } finally {
+     setSending(false);
+   }
+ }
+
+ return (
+   <section className="container" style={{ padding: "48px 24px 96px", maxWidth:
640 }}>
+     <p className="eyebrow" style={{ marginBottom: 12 }}>Wallet</p>
+     <h1 style={{ fontSize: 30, marginBottom: 8 }}>Your D3RAC balance</h1>
+     <p style={{ color: "var(--text-muted)", marginBottom: 32 }}>
+       Connected to <strong style={{ color: "var(--text)" }}>{adapter.label}
</strong>
+     {chainId === "tron" && (
+       <>
+         { " " }
+       <span className="mono">{currentTronNetwork()}</span>
+     </>

```

```

+     }}
+     . Switch chains from the selector in the top bar.
+ </p>
+
+     {!!address ? (
+         <div className="card" style={{ textAlign: "center" }}>
+             <p style={{ marginBottom: 16, color: "var(--text-muted)" }}>
+                 Connect a {adapter.label} wallet to see your balance and send D3RAC.
+             </p>
+             {adapter.isWalletAvailable() ? (
+                 <button className="btn btn-primary" onClick={connect} disabled=
+ {connecting}>
+                 {connecting ? "Connecting..." : `Connect ${adapter.label} wallet`}
+             </button>
+             ) : (
+                 <a href={adapter.installUrl} target="_blank" rel="noreferrer"
+ className="btn btn-primary">
+                 Install {adapter.label} wallet
+             </a>
+             )}
+         </div>
+     ) : (
+         <>
+             <div className="card" style={{ marginBottom: 20 }}>
+                 <p style={{ fontSize: 13, color: "var(--text-muted)", marginBottom: 6
+ }}>Your address</p>
+                 <div style={{ display: "flex", alignItems: "center", gap: 10,
+ flexWrap: "wrap" }}>
+                     <span className="mono" style={{ fontSize: 14, wordBreak: "break-
+ all" }}>{address}</span>
+                     <button type="button" className="btn btn-ghost" onClick=
+ {copyAddress} style={{ padding: "4px 10px", fontSize: 12 }}>
+                         {copied ? "Copied" : "Copy"}
+                     </button>
+                     <a
+                         href={adapter.explorerAddressUrl(address)}
+                         target="_blank"
+                         rel="noreferrer"
+                         style={{ fontSize: 12, color: "var(--indigo)", textDecoration:
+ "underline" }}
+                     >
+                         View on explorer →
+                     </a>
+                 </div>
+
+                 <div style={{ display: "flex", gap: 32, marginTop: 20, flexWrap:
+ "wrap" }}>

```

```

+         <div>
+             <p style={{ fontSize: 12, color: "var(--text-muted)",
marginBottom: 4 }}>{adapter.nativeSymbol} (network fees)</p>
+             <p className="mono" style={{ fontSize: 20, color: "var(--text)"
}}>
+                 {loadingBalances && nativeBalance === null ? "...":
(nativeBalance ?? "-")}
+             </p>
+         </div>
+         <div>
+             <p style={{ fontSize: 12, color: "var(--text-muted)",
marginBottom: 4 }}>D3RAC</p>
+             {tokenAddress ? (
+                 <p className="mono" style={{ fontSize: 20, color: "var(--teal)"
}}>
+                     {loadingBalances && tokenBalance === null ? "...":
(tokenBalance ? tokenBalance.amount : "-")}
+                 </p>
+             ) : (
+                 <p style={{ fontSize: 13, color: "var(--text-faint)" }}>
+                     Not deployed on {chainId === "tron" ? currentTronNetwork() :
"this network"} yet
+                 </p>
+             )}
+         </div>
+     </div>
+
+     {balanceErr && <p style={{ color: "var(--coral)", fontSize: 13,
marginTop: 12 }}>{balanceErr}</p>}
+ </div>
+
+     {tokenAddress && (
+         <form onSubmit={handleSend} className="card" style={{ display:
"flex", flexDirection: "column", gap: 18, marginBottom: 20 }}>
+             <p style={{ fontSize: 13, color: "var(--text-muted)", margin: 0
}}>Send D3RAC</p>
+
+             <Field label="Recipient address">
+                 <input value={to} onChange={(e) => setTo(e.target.value)}
required style={inputStyle} placeholder={chainId === "tron" ? "T..." : "hash-..."}
/>
+             </Field>
+
+             <Field label="Amount">
+                 <input value={amount} onChange={(e) => setAmount(e.target.value)}
type="number" step="any" min="0" required style={inputStyle} />
+             </Field>

```



```

+
+         <button type="submit" className="btn btn-primary" disabled=
{sending}>
+             {sending ? "Sending..." : "Send"}
+         </button>
+
+         {sendErr && <p style={{ color: "var(--coral)", fontSize: 13 }}>
{sendErr}</p>}
+         {sendResult && (
+             <p style={{ color: "var(--teal)", fontSize: 13 }}
className="mono">
+                 Sent.{" "}
+                 <a href={sendResult.explorerUrl} target="_blank"
rel="noreferrer" style={{ textDecoration: "underline" }}>
+                     View on explorer →
+                 </a>
+             </p>
+         )}
+     </form>
+ })
+
+     <div className="card">
+         <p style={{ fontSize: 13, color: "var(--text-muted)", marginBottom:
12 }}>
+             Prefer to donate directly instead of through the app? These are
D3R·AC's official addresses
+             (see docs/donations.md) – always double-check the network label
before sending.
+         </p>
+         <div style={{ display: "flex", flexDirection: "column", gap: 10 }}>
+             {DONATION_ADDRESSES.map((d) => (
+                 <div key={d.address} style={{ display: "flex", flexDirection:
"column", gap: 2 }}>
+                     <span style={{ fontSize: 12, color: d.isDefault ? "var(--
amber)" : "var(--text-muted)" }}>
+                         {d.label}
+                         {d.isDefault ? " (default)" : ""}
+                     </span>
+                     <span className="mono" style={{ fontSize: 13, wordBreak:
"break-all" }}>{d.address}</span>
+                 </div>
+             )})
+         </div>
+     </div>
+ </>
+ })
+ </section>

```

+) ;

+ }